

✓ Preparing Data

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
# preparing the data
# from the link ( https://www.cars.com/research/ford-ranger/ )

years = [
    2026, 2025, 2024, 2023, 2022, 2021, 2020, 2019,
    2011, 2010, 2009, 2008, 2007, 2006, 2005, 2004,
    2003, 2002, 2001, 2000, 1999, 1998
]

minPrice = [
    33550, 33550, 32820, 27400, 25980, 24820, 24110,
    24000, 18160, 17820, 16395, 14490, 13970, 14450,
    14610, 14575, 13645, 12565, 11960, 11580, 11845, 11485
]

maxPrice = [
    57270, 56070, 55720, 40945, 39730, 39035, 38675,
    38565, 26070, 25800, 25805, 24350, 24425, 26670,
    26795, 26015, 25450, 25010, 24335, 19785, 19435, 19695
]
```

```
# joint those three data list into the panda data frame
data_df = pd.DataFrame({
    'year':years,
    'min Price':minPrice,
    'max Price':maxPrice
})
# check if the data match correctly
display(data_df)
```

	year	min Price	max Price	
0	2026	33550	57270	
1	2025	33550	56070	
2	2024	32820	55720	
3	2023	27400	40945	
4	2022	25980	39730	
5	2021	24820	39035	
6	2020	24110	38675	
7	2019	24000	38565	
8	2011	18160	26070	
9	2010	17820	25800	
10	2009	16395	25805	
11	2008	14490	24350	
12	2007	13970	24425	
13	2006	14450	26670	
14	2005	14610	26795	
15	2004	14575	26015	
16	2003	13645	25450	
17	2002	12565	25010	
18	2001	11960	24335	
19	2000	11580	19785	
20	1999	11845	19435	
21	1998	11485	19695	

Next steps:

[Generate code with data_df](#)[New interactive sheet](#)

✓ Preparing Model Algorithm

```
# the linear regression algorithm
# function is  $h(x) = \theta_0 + \theta_1 * x$ 
#
# SIGNATURES:
# x: features of the data
# theta0: the value of theta 0 of linear regression(h(x))
# theta1: the value of theta 1 of linear regression(h(x))
#
```

```

def linear_regression(x, theta0, theta1):
    return theta0 + theta1 * x

# This function used to test how the loss changed
# cost function use the error and loss function
# function is  $(1/m) * (\sum(e_i^2 + 0.1 * e_i^4))$  (where i is integer from 1 to m)
# where  $e_i = h(x_i) - y_i$ 
#
# SIGNATURES:
# data_df: data set recording the year and related vehicles price of mi
# price_col: the name of the price column in data_df
# year_col: the name of the year column in data_df
# theta0: the value of theta 0 of linear regression
# theta1: the value of theta 1 of linear regression
#
def cost_func(data_df, price_col, year_col, theta0, theta1):
    my_years = data_df[year_col]
    my_price = data_df[price_col]

    errorSum = 0
    for i in range(len(my_years)):
        year = my_years.iloc[i]
        actual_price = my_price.iloc[i]
        prediction = linear_regression(year, theta0=theta0, theta1=theta1)
        error = prediction - actual_price
        errorSum += (error**2 + 0.1*error**4)
    return errorSum/len(my_years)

```

Now, I need to calculate the Gradient. The gradient function should be the derivative of cost function, and its derivation formula is shown in the picture below

$$h(x) = \theta_0 + \theta_1 x$$

$$e^{(i)} = h(x^{(i)}) - y^{(i)}$$

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m ((e^{(i)})^2 + 0.1 (e^{(i)})^4)$$

$$\frac{\partial e}{\partial \theta_0} = \frac{\partial}{\partial \theta_0} (h(x) - y)$$

$$= \frac{\partial}{\partial \theta_0} (\theta_0 + \theta_1 x - y)$$

$$= 0 + 1 - 0 = 1$$

$$\frac{\partial e}{\partial \theta_1} = \frac{\partial}{\partial \theta_1} (h(x) - y)$$

$$= \frac{\partial}{\partial \theta_1} (\theta_0 + \theta_1 x - y)$$

$$= 0 + x - 0 = x$$

$$\frac{\partial J}{\partial e} = \frac{\partial}{\partial e} (e^2 + 0.1 e^4)$$

$$= 2e + 0.4e^3$$

$$\text{Gradient} = \frac{\partial J}{\partial \theta_0}, \frac{\partial J}{\partial \theta_1}$$

$$\frac{\partial J}{\partial \theta_0} = \frac{\partial J}{\partial e} \cdot \frac{\partial e}{\partial \theta_0}$$

$$= (2e + 0.4e^3)(1)$$

$$= 2e + 0.4e^3$$

$$= 2(h(x) - y) + 0.4(h(x) - y)^3$$

$$= 2(\theta_0 + \theta_1 x - y) + 0.4(\theta_0 + \theta_1 x - y)^3$$

$$\frac{\partial J}{\partial \theta_1} = \frac{\partial J}{\partial e} \cdot \frac{\partial e}{\partial \theta_1}$$

$$= (2e + 0.4e^3)(x)$$

$$= (2e + 0.4e^3)x$$

$$= [2(\theta_0 + \theta_1 x - y) + 0.4(\theta_0 + \theta_1 x - y)^3] \cdot x$$

```

#
# Gradient function based on the cost function J(theta)
# this is the derivative of the cost Function delta(J)/delta(e)
#
# SIGNATURES:
# data_df: data set recording the year and related vehicles price of mi
# price_col: the name of the price column in data_df
# year_col: the name of the year column in data_df
# theta0: the value of theta 0 of linear regression
# thetal: the value of theta 1 of linear regression
#
# return: a tuple store the value of gradient theta 0 and gradient the
#
def gradient(data_df, price_col, year_col, theta0, thetal):
    my_years = data_df[year_col]
    my_price = data_df[price_col]

    grad_theta0 = 0
    grad_thetal = 0
    for i in range(len(my_years)):
        year = my_years.iloc[i]

        actual_price = my_price.iloc[i]
        prediction = linear_regression(year, theta0=theta0, thetal=thetal)
        error = prediction - actual_price
        grad_theta0 += (2*error + 0.4*error**3)
        grad_thetal += (2*error + 0.4*error**3)*year
    return (grad_theta0/len(my_years), grad_thetal/len(my_years))

```

Here is the gradient descent formula based on the gradient

```

# Function to calculate the gradient descent
# the main formula is new_theta = old_theta - Alpha * gradient(theta)
#
# SIGNATURES:
# data_df: data set recording the year and related vehicles price of mi
# price_col: the name of the price column in data_df
# year_col: the name of the year column in data_df
# theta0: the value of theta 0 of linear regression
# thetal: the value of theta 1 of linear regression
# alpha: learning rate
#
# return: returns a tuple of new theta 0 and new theta 1
#
def gradient_descent(data_df, price_col, year_col, theta0, thetal, alpha):
    grad_theta0, grad_thetal = gradient(data_df, price_col, year_col, theta0, thetal)
    new_theta0 = theta0 - alpha*grad_theta0
    new_thetal = thetal - alpha*grad_thetal
    return (new_theta0, new_thetal)

```

✓ Learning Test

Here is the test of updating the theta 1 time because the cost function is exponent so I just want to see what the update value of theta and cost are when I initial the theta value as (0,0)

```
init_theta0 = 0
init_theta1 = 0

init_predict = linear_regression(years[0], theta0=init_theta0, theta1=init_theta1)
init_grad_theta0, init_grad_theta1 = gradient(data_df, 'min Price', 'year', in
init_cost = cost_func(data_df, 'min Price', 'year', init_theta0, init_theta1)

print(f"start prediction: {init_predict}")
print(f"start gradient of theta 0: {init_grad_theta0}")
print(f"start gradient of theta 1: {init_grad_theta1}")
print(f"start cost function: {init_cost}")
#
```

```
start prediction: 0
start gradient of theta 0: -4257003258466.364
start gradient of theta 1: -8600508450284192.0
start cost function: 2.907286085483342e+16
```

```
test_alpha = 0.0001
test_new_theta0, test_new_theta1 = gradient_descent(data_df, 'min Price', 'year
print(f"test new theta 0: {test_new_theta0}")
print(f"test new theta 1: {test_new_theta1}")
new_predict = linear_regression(years[0], theta0=test_new_theta0, theta1=test_new
print(f"test new prediction: {new_predict}")
```

```
test new theta 0: 425700325.8466364
test new theta 1: 860050845028.4192
test new prediction: 1742463437727903.0
```

Here since the cost function is exponential, even I give the 0.0001 as the learning rate the theta value and the prediction is still too big, in this case I need to provide a smaller value for learning rate

✓ Traditional Learning

The regular learning with the fixed value of learning rate and original value of feature(years)

The output here are stored into the list given into the signatures showing below those output signature are used to store the values instead of return all them.

```
# learning algorithm used to doing the gradient descent to update the weight(theta)
#
# SIGNITURES INPUT:
# learning_rate: (float type) learning rate(alpha) of the update function
# iteration_num: (Integer type) num of iteration
# theta: (a tuple) store the theta 0 and theta 1
# data_df: the panda data frame that stores the years, min price, and max price
# price_col: (String type) a name of the price column of data frame (min price / max
# year_col: (String type) a name of the feature column(year) of panda data frame
#
# SIGNITURES OUTPUT:
# predicct_list: 2-d array to store the hyphothesis result h(x) for every iteration
# theta0_list: a list to store the update value of theta 0 for every iteration
# cost_list: a list to store
#
# Return: void
#
def learning(learning_rate, iteration_num, theta, data_df, price_col, year_col,
             predicts_list, theta0_list, thetal_list, cost_list):
    new_theta0, new_thetal = theta
    for i in range(iteration_num):
        new_theta0, new_thetal = gradient_descent(data_df, price_col, year_col, new_theta0, new_thetal)
        theta0_list.append(new_theta0)
        thetal_list.append(new_thetal)

        cost = cost_func(data_df, price_col, year_col, new_theta0, new_thetal)
        cost_list.append(cost)

        predict_list = []
        for j in range(len(years)):
            predict_list.append(linear_regression(years[j], theta0=new_theta0, thetal=new_thetal))
        predicts_list.append(predict_list)

    # prepare the value of learning rate(alpha) and the iteration number of how many time
    alpha = 1e-16
    iteration_num = 100
```

✓ Minimum Modeling

Here is how I build the Model of Minimum price of the vehicles

```
min_theta = (0,0)
min_hypothesis_list = []
min_theta0_list = [min_theta[0]]
min_thetal_list = [min_theta[1]]
min_cost_list = []

learning(alpha, iteration_num, min_theta, data_df, 'min Price', 'year',
        min_hypothesis_list, min_theta0_list, min_thetal_list, min_cost_list)

# get the last iteration result of prediciton for the min price
min_final_hypothesis = min_hypothesis_list[len(min_hypothesis_list)-1]
# get the last iteration result of theta 0
min_final_theta0 = min_theta0_list[len(min_theta0_list)-1]
# get the last iteration result of theta 1
min_final_thetal = min_thetal_list[len(min_thetal_list)-1]
# a list to store the error of prediction from the last iteration
min_final_error = []

# calculate the error and store the error into the min_final_error list
for i in range(len(years)):
    min_final_error.append(round((min_final_hypothesis[i] - minPrice[i]),2))

print(f"final modeling result (linear regression result):"+
      f" h(x) = {round(min_final_theta0,5)} + {round(min_final_thetal,5)} * x")
print(f"value of cost function of last prediction: {round(min_cost_list[len(min_cost_list)-1],

# build the data frame to see the performance of my prediciton
min_result_df = pd.DataFrame({
    "Years":years,
    "Min Price":minPrice,
    "Predicts":min_final_hypothesis,
    "Error":min_final_error,
})

display((min_result_df))
```

final modeling result (linear regression result): $h(x) = 0.00496 + 10.04571 * x$
 value of cost function of last prediction: 544708182093170.06

	Years	Min Price	Predicts	Error	
0	2026	33550	20352.611642	-13197.39	
1	2025	33550	20342.565933	-13207.43	
2	2024	32820	20332.520224	-12487.48	
3	2023	27400	20322.474514	-7077.53	
4	2022	25980	20312.428805	-5667.57	
5	2021	24820	20302.383096	-4517.62	
6	2020	24110	20292.337387	-3817.66	
7	2019	24000	20282.291678	-3717.71	
8	2011	18160	20201.926005	2041.93	
9	2010	17820	20191.880296	2371.88	
10	2009	16395	20181.834587	3786.83	
11	2008	14490	20171.788878	5681.79	
12	2007	13970	20161.743169	6191.74	
13	2006	14450	20151.697459	5701.70	
14	2005	14610	20141.651750	5531.65	
15	2004	14575	20131.606041	5556.61	
16	2003	13645	20121.560332	6476.56	
17	2002	12565	20111.514623	7546.51	
18	2001	11960	20101.468914	8141.47	
19	2000	11580	20091.423205	8511.42	
20	1999	11845	20081.377496	8236.38	
21	1998	11485	20071.331786	8586.33	

Next steps:

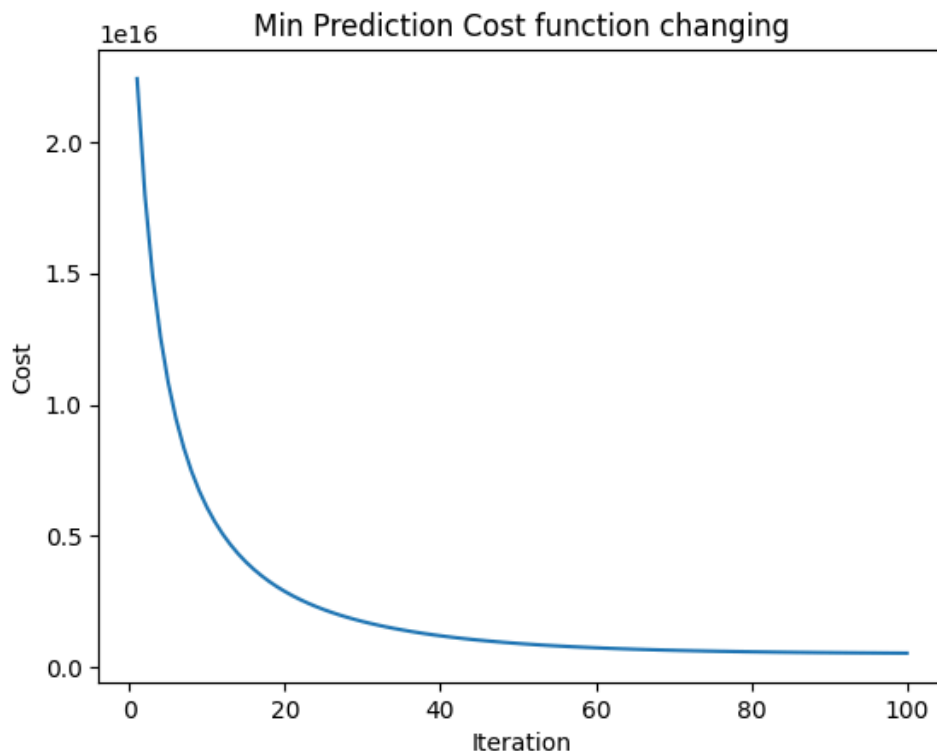
[Generate code with min_result_df](#)

[New interactive sheet](#)

```
#
# draw the figure of cost function to see how does the
# cost value changed during the 100 iterations
#
plt.plot(range(1, iteration_num + 1), min_cost_list)

plt.xlabel("Iteration")
plt.ylabel("Cost")
plt.title("Min Prediction Cost function changing")

plt.show()
```



we can see here the Cost curve shows the value has converge at the end of the iteration and the final theta numbers for the minimum price are:

1. theta 0 = 0.00496
2. theta 1 = 10.04571

However, prediction errors shown in data frame are still big

✓ Prediction of minimum price from 2012 to 2018

Here is the prediction of the minimum price of vehicles from year 2012 to 2018

Here I just shows the plot from 2012 to 2018 the final plot integrate with all data are shown at the end of this Part

```
predict_years = [2012, 2013, 2014, 2015, 2016, 2017, 2018]

# a list to store the prediction price of the minimum value
min_price_predict = []
# doing the prediction based on the model
for year in predict_years:
    min_price_predict.append(linear_regression(year, theta0=min_final_theta0, theta1=min_final_theta1))

# build the data frame to see the result of prediction
predict_min_df = pd.DataFrame({
    "Years":predict_years,
    "Predicts":min_price_predict
})
display(predict_min_df)
```

	Years	Predicts	
0	2012	20211.971714	
1	2013	20222.017423	
2	2014	20232.063132	
3	2015	20242.108841	
4	2016	20252.154551	
5	2017	20262.200260	
6	2018	20272.245969	

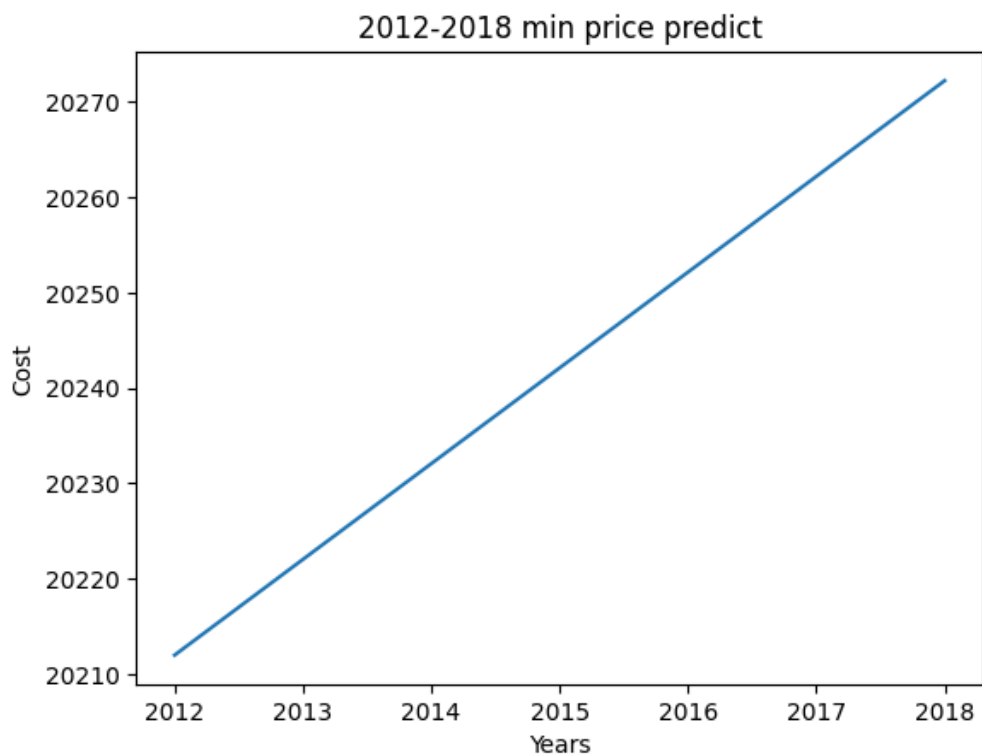
Next steps:

[Generate code with predict_min_df](#)[New interactive sheet](#)

```
# build the plot to see of the price changed based on the year
plt.plot(range(2012, 2019), min_price_predict)

plt.xlabel("Years")
plt.ylabel("Cost")
plt.title("2012-2018 min price predict")

plt.show()
```



✓ Max Modeling

Here is how I build the Model to predict the maximum price of the vehicles based on the learning data

```
max_theta = (0,0)
max_hypothesis_list = []
max_theta0_list = [max_theta[0]]
max_theta1_list = [max_theta[1]]
max_cost_list = []
```

```

# doing the learning to update the maximum modeling
learning(alpha, iteration_num, max_theta, data_df, 'max Price', 'year', max_hypothesis_list,
# get the last iteration result of prediciton for the max price
max_final_hypothesis = max_hypothesis_list[len(max_hypothesis_list)-1]
# get the last iteration result of the theta0
max_final_theta0 = max_theta0_list[len(max_theta0_list)-1]
# get the last iteration result of the thetal
max_final_thetal = max_thetal_list[len(max_thetal_list)-1]
# a list to store the errors of prediction from the max_final_hypothesis
max_final_error = []

# calculate the error and store the error into the max_final_error list
for i in range(len(years)):
    max_final_error.append(round((max_final_hypothesis[i] - maxPrice[i]),2))

print(f"final modeling result (linear regression result):"+
      f" h(x) = {round(max_final_theta0,5)} + {round(max_final_thetal,5)} * x")
print(f"value of cost function of last prediction: {round(max_cost_list[len(max_cost_list)-1],
# build the data frame to see the performance of my prediciton
max_result_df = pd.DataFrame({
    "Years":years,
    "Max Price":maxPrice,
    "Predicts":max_final_hypothesis,
    "Error":max_final_error,
}))

display((max_result_df))

```

final modeling result (linear regression result): $h(x) = 0.0088 + 17.8791 * x$
 value of cost function of last prediction: 3777515622392022.5

	Years	Max Price	Predicts	Error	
0	2026	57270	36223.056590	-21046.94	
1	2025	56070	36205.177495	-19864.82	
2	2024	55720	36187.298399	-19532.70	
3	2023	40945	36169.419304	-4775.58	
4	2022	39730	36151.540208	-3578.46	
5	2021	39035	36133.661112	-2901.34	
6	2020	38675	36115.782017	-2559.22	
7	2019	38565	36097.902921	-2467.10	
8	2011	26070	35954.870156	9884.87	
9	2010	25800	35936.991060	10136.99	
10	2009	25805	35919.111964	10114.11	
11	2008	24350	35901.232869	11551.23	
12	2007	24425	35883.353773	11458.35	
13	2006	26670	35865.474677	9195.47	
14	2005	26795	35847.595582	9052.60	
15	2004	26015	35829.716486	9814.72	
16	2003	25450	35811.837391	10361.84	
17	2002	25010	35793.958295	10783.96	
18	2001	24335	35776.079199	11441.08	
19	2000	19785	35758.200104	15973.20	
20	1999	19435	35740.321008	16305.32	
21	1998	19695	35722.441912	16027.44	

Next steps:

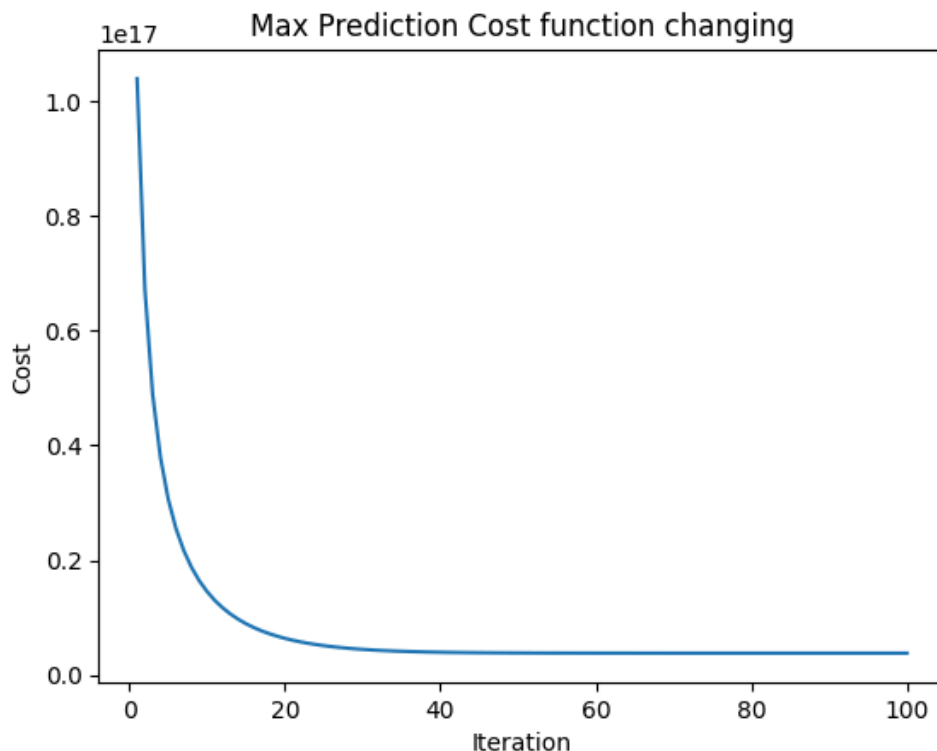
[Generate code with max_result_df](#)

[New interactive sheet](#)

```
#
# draw the figure of cost function to see how does the
# cost value changed during the 100 iterations
#
plt.plot(range(1, iteration_num + 1), max_cost_list)

plt.xlabel("Iteration")
plt.ylabel("Cost")
plt.title("Max Prediction Cost function changing")

plt.show()
```



The cost value during the iteration seems also convergence at the end of the iteration, which is what we want, and the theta value for the maximum price are:

1. theta 0 = 0.0088
2. theta 1 = 17.8791

✓ Prediction of maximum price from 2012 to 2018

Here is how I do the prediction of max price of vehicles from the year 2012 to 2018

```
# a list to store the prediction result
max_price_predict = []
# prediction process based on the result of theta 0 and theta 1 from learning last it
for year in predict_years:
    max_price_predict.append(linear_regression(year, theta0=max_final_theta0, theta1=max_final_thet

# build the data frame to see the result of prediction
predict_min_df = pd.DataFrame({
    "Years":predict_years,
    "Predicts":max_price_predict
})
display(predict_min_df)
```

	Years	Predicts	
0	2012	35972.749251	
1	2013	35990.628347	
2	2014	36008.507443	
3	2015	36026.386538	
4	2016	36044.265634	
5	2017	36062.144730	
6	2018	36080.023825	

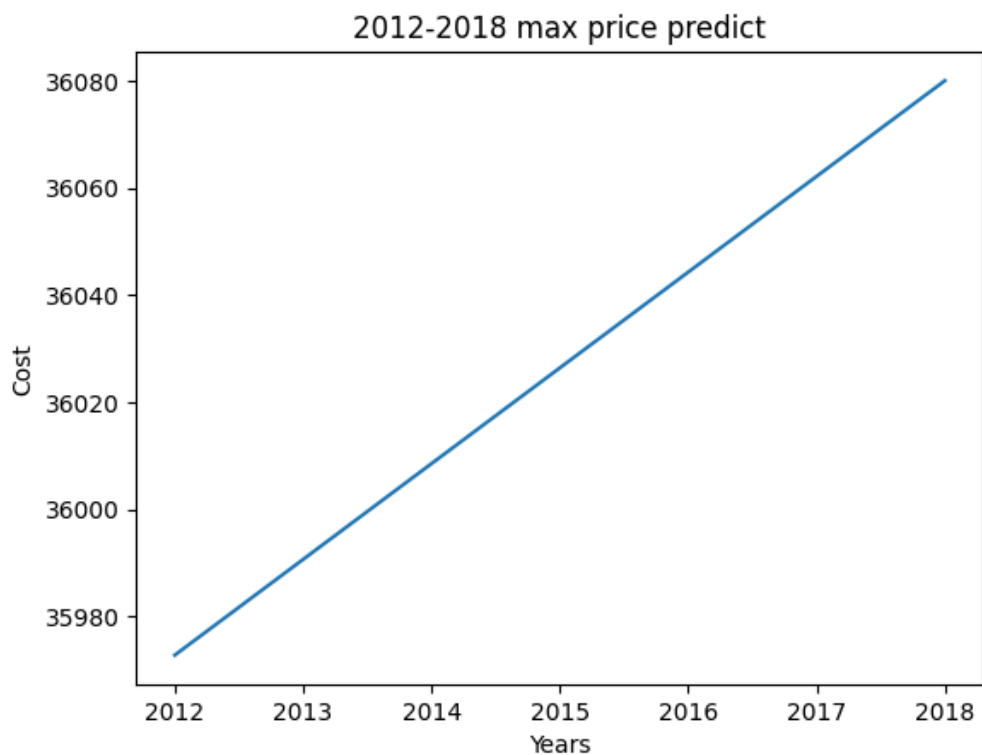
Next steps:

[Generate code with predict_min_df](#)[New interactive sheet](#)

```
# draw the plot to see how does the predicition of max price change
plt.plot(range(2012, 2019), max_price_predict)

plt.xlabel("Years")
plt.ylabel("Cost")
plt.title("2012-2018 max price predict")

plt.show()
```



✓ Year-to-Price Curves with predicted values for missing years

```
predict_df = pd.DataFrame({
    "year": predict_years,
    "min Price": min_price_predict,
    "max Price": max_price_predict
})
# add the prediction into the original frame
new_data_df = pd.concat(
    [data_df[["year", "min Price", "max Price"]], predict_df],
```

```
ignore_index=True
)

new_data_df = new_data_df.sort_values(
    by="year"
).reset_index(drop=True)

display(new_data_df)
```

	year	min Price	max Price	
0	1998	11485.000000	19695.000000	 
1	1999	11845.000000	19435.000000	
2	2000	11580.000000	19785.000000	
3	2001	11960.000000	24335.000000	
4	2002	12565.000000	25010.000000	
5	2003	13645.000000	25450.000000	
6	2004	14575.000000	26015.000000	
7	2005	14610.000000	26795.000000	
8	2006	14450.000000	26670.000000	
9	2007	13970.000000	24425.000000	
10	2008	14490.000000	24350.000000	
11	2009	16395.000000	25805.000000	
12	2010	17820.000000	25800.000000	
13	2011	18160.000000	26070.000000	
14	2012	20211.971714	35972.749251	
15	2013	20222.017423	35990.628347	
16	2014	20232.063132	36008.507443	
17	2015	20242.108841	36026.386538	
18	2016	20252.154551	36044.265634	
19	2017	20262.200260	36062.144730	
20	2018	20272.245969	36080.023825	
21	2019	24000.000000	38565.000000	
22	2020	24110.000000	38675.000000	
23	2021	24820.000000	39035.000000	
24	2022	25980.000000	39730.000000	
25	2023	27400.000000	40945.000000	
26	2024	32820.000000	55720.000000	
27	2025	33550.000000	56070.000000	
28	2026	33550.000000	57270.000000	

Next steps:

[Generate code with new_data_df](#)[New interactive sheet](#)

```
plt.plot(
    new_data_df["year"],
```

```

        new_data_df["min Price"],
        label="Min Price"
    )

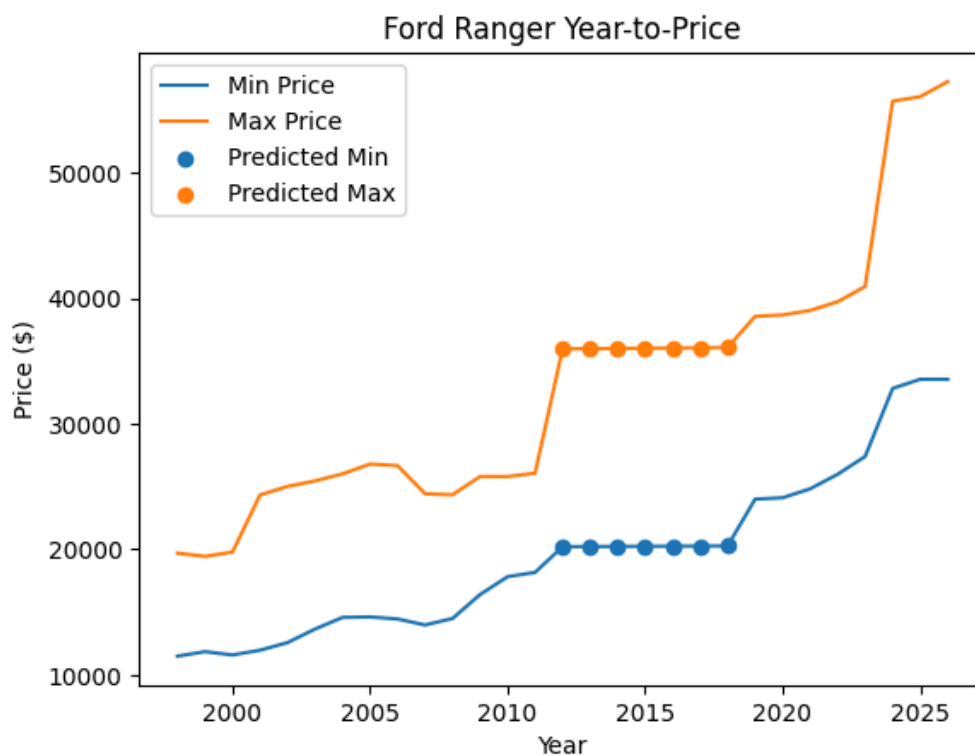
    plt.plot(
        new_data_df["year"],
        new_data_df["max Price"],
        label="Max Price"
    )

    plt.scatter(
        predict_years,
        min_price_predict,
        label="Predicted Min"
    )

    plt.scatter(
        predict_years,
        max_price_predict,
        label="Predicted Max"
    )

    plt.xlabel("Year")
    plt.ylabel("Price ($)")
    plt.title("Ford Ranger Year-to-Price")
    plt.legend()
    plt.show()

```



The part marked by points are the part that I predict based on the Model Theta, and we can see here it is kind flat.

✓ Feature Scaling

Here is the modeling with feature scaling and dynamic learning rate

```
# a function to scale the feature(years) from the data sets
#
# here are two type of the scaling:
# type(1) (x-miu)/standard deviation
# type(2) (x-miu)/Range of value
#
# SIGNATURES:
# features: feature of data set (years in this case)
# scaling_type: the type of the scaling:
#     1) (int) 1 represent type-1
#     2) (int) 2 represent type-2
#     3) others raise the exception
#
# Returns: a list of scaling freatures
#
def feature_scaling(features, scaling_type):
    result = []
    miu = np.mean(features)

    match scaling_type:
        case 1:
            for feature in features:
                result.append((feature - miu)/np.std(features))
        case 2:
            max_value = np.max(features)
            min_value = np.min(features)
            for feature in features:
```

```
        result.append((feature - miu)/(max_value - min_value))
    case _:
        raise Exception("Invalid scaling type there are two type (1 or
return result
```

Here I just joint the scaling value into the data set for doing the modeling later

```
# scaling feature with the type 1
scaling_result1 = feature_scaling(years, 1)
# scaling feature with the type 2
scaling_result2 = feature_scaling(years, 2)

# join the scaling features into the data set
data_df['scaling1'] = scaling_result1
data_df['scaling2'] = scaling_result2
# check the data set
display(data_df)
```

	year	min Price	max Price	scaling1	scaling2	
0	2026	33550	57270	1.601224	0.534091	
1	2025	33550	56070	1.494152	0.498377	
2	2024	32820	55720	1.387079	0.462662	
3	2023	27400	40945	1.280006	0.426948	
4	2022	25980	39730	1.172933	0.391234	
5	2021	24820	39035	1.065861	0.355519	
6	2020	24110	38675	0.958788	0.319805	
7	2019	24000	38565	0.851715	0.284091	
8	2011	18160	26070	-0.004867	-0.001623	
9	2010	17820	25800	-0.111940	-0.037338	
10	2009	16395	25805	-0.219012	-0.073052	
11	2008	14490	24350	-0.326085	-0.108766	
12	2007	13970	24425	-0.433158	-0.144481	
13	2006	14450	26670	-0.540231	-0.180195	
14	2005	14610	26795	-0.647304	-0.215909	
15	2004	14575	26015	-0.754376	-0.251623	
16	2003	13645	25450	-0.861449	-0.287338	
17	2002	12565	25010	-0.968522	-0.323052	
18	2001	11960	24335	-1.075595	-0.358766	
19	2000	11580	19785	-1.182667	-0.394481	
20	1999	11845	19435	-1.289740	-0.430195	
21	1998	11485	19695	-1.396813	-0.465909	

Next steps:

[Generate code with data_df](#)[New interactive sheet](#)

```

# a function to calculate how the learning rate changed based on
# the current iteration number
# Formular: new alpha = old alpha * (constant / (constant + tau))
# (where the tau is the current iteration number)
#
# SIGNATURES:
# learning_rate: the initial value of the learning rate
# iteration_num: current number of iteration
# constant: a constant number in dynamic learning rate formular
#
# returns: a float value
#

```

```

def dynamic_Learning_rate(learning_rate, iteration_num, constant):
    new_alpha = learning_rate*(constant/(constant+iteration_num))
    return new_alpha

# dynamic learning algorithm used to doing the gradient descent
# with dynamic learning rate to update the weight(theta)
#
# SIGNITURES INPUT:
# learning_rate: (float type) learning rate(alpha) of the update function
# data_df: the panda data frame that stores the years, min price, and max price
# iteration_num: (Integer type) num of iteration
# theta: (a tuple) store the initial theta 0 and theta 1
# constant: a constant number to calculate the dynamic learning rate
# price_col: (String type) a name of the column name of the panda data
# year_col: name of the of data_df
#
# SIGNITURES OUTPUT:
# alpha_list: a list to store the update value of alpha for every iteration
# predicts_list: 2-d array to store the hyphotesis result h(x) for every iteration
# theta0_list: a list to store the update value of theta 0 for every iteration
# thetal_list: a list to store the update value of theta 1 for every iteration
# cost_list: a list to store the value of cost function
#
# Return: void
def dynamic_learning(learning_rate, data_df, iteration_num, theta, constant, alpha_list, predicts_list, theta0_list, thetal_list, cost_list):
    new_theta0, new_thetal = theta
    new_alpha = learning_rate
    for i in range(iteration_num):
        new_alpha = dynamic_Learning_rate(learning_rate, i, constant)
        alpha_list.append(new_alpha)
        new_theta0, new_thetal = gradient_descent(data_df, price_col, year_col, new_alpha, new_theta0, new_thetal)
        theta0_list.append(new_theta0)
        thetal_list.append(new_thetal)

    cost = cost_func(data_df, price_col, year_col, new_theta0, new_thetal)
    cost_list.append(cost)

    features = data_df[year_col]
    predict_list = []
    for j in range(len(years)):
        predict_list.append(linear_regression(features[j], theta0=new_theta0, theta1=new_thetal))
    predicts_list.append(predict_list)

# define the initial value of learning rate
learning_rate = 2e-9
# define the value of iteration number
iteration_num = 100
# define the initial value of theta
theta = (0,0)
# define the value of constant c for dynamic learning rate
constant = 10
# define the type of scaling that I use

```

```
year_coll = 'scaling'
```

✓ minimum modeling for feature scaling 1

I implemented two feature-scaling methods, but I used the first method, standardization for the scaled models

$(x - \mu) / \text{standard deviation}$,

```
# column name of data_df
min_price_col = 'min Price'
min_alpha_list = []
min_hypothesis_list1 = []
min_theta0_list1 = [theta[0]]
min_theta1_list1 = [theta[1]]
min_cost_list1 = []

# learning
dynamic_learning(learning_rate, data_df, iteration_num, theta, constant, min_p
                  min_alpha_list, min_hypothesis_list1, min_theta0_list1, 1

# a list to store the values of the learning rate during the iteration
min_final_hypothesis1 = min_hypothesis_list1[len(min_hypothesis_list1)-1]
# the value of theta 0 from the last iteration
min_final_theta01 = min_theta0_list1[len(min_theta0_list1)-1]
# the value of theta 1 from the last iteration
min_final_theta11 = min_theta1_list1[len(min_theta1_list1)-1]
# a list to store the error from the prediction min_final_hypothesis1
min_final_error1 = []

# calculate the error and store the error into the min_final_error list
for i in range(len(years)):
    min_final_error1.append(round((min_final_hypothesis1[i] - minPrice[i]),5))

print(f"final modeling result (linear regression result):"+
      f" h(x) = {round(min_final_theta01,5)} + {round(min_final_theta11,5)} * x"
print(f"value of cost function of last prediction: {round(min_cost_list1[len

# build the data frame to see the performance of my prediciton
min_result_df1 = pd.DataFrame({
    "Years":years,
    "Min Price":minPrice,
    "Predicts":min_final_hypothesis1,
    "Error":min_final_error1,
})

display((min_result_df1))
```

final modeling result (linear regression result): $h(x) = 15209.70501 + 7532.72664 * x$
 value of cost function of last prediction: 65323174689302.125

	Years	Min Price	Predicts	Error	
0	2026	33550	27271.291092	-6278.70891	
1	2025	33550	26464.741263	-7085.25874	
2	2024	32820	25658.191433	-7161.80857	
3	2023	27400	24851.641604	-2548.35840	
4	2022	25980	24045.091774	-1934.90823	
5	2021	24820	23238.541945	-1581.45806	
6	2020	24110	22431.992115	-1678.00788	
7	2019	24000	21625.442286	-2374.55771	
8	2011	18160	15173.043650	-2986.95635	
9	2010	17820	14366.493821	-3453.50618	
10	2009	16395	13559.943991	-2835.05601	
11	2008	14490	12753.394162	-1736.60584	
12	2007	13970	11946.844332	-2023.15567	
13	2006	14450	11140.294503	-3309.70550	
14	2005	14610	10333.744673	-4276.25533	
15	2004	14575	9527.194844	-5047.80516	
16	2003	13645	8720.645014	-4924.35499	
17	2002	12565	7914.095185	-4650.90482	
18	2001	11960	7107.545355	-4852.45464	
19	2000	11580	6300.995526	-5279.00447	
20	1999	11845	5494.445696	-6350.55430	
21	1998	11485	4687.895867	-6797.10413	

Next steps:

[Generate code with min_result_df1](#)

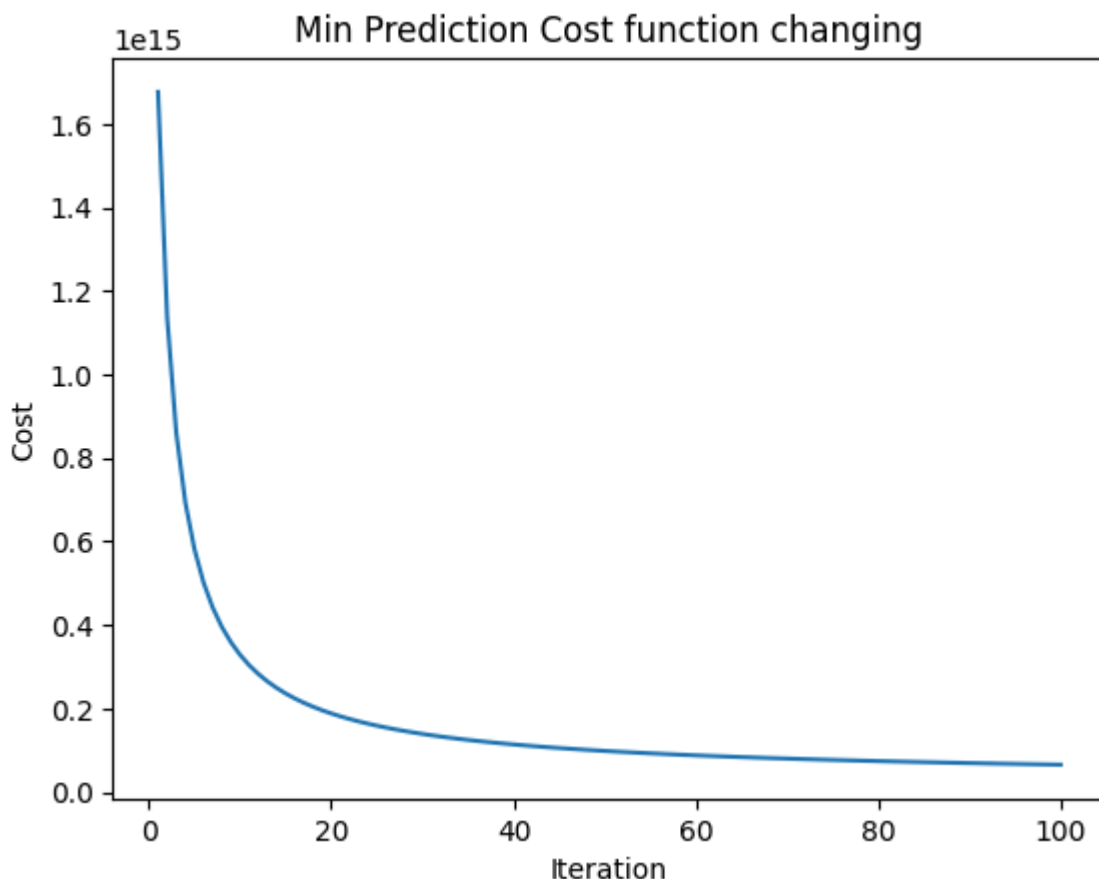
[New interactive sheet](#)

```
#
# draw the figure of cost function to see how does the
# cost value changed during the 100 iterations
#
plt.plot(range(1, iteration_num + 1), min_cost_list1)

plt.xlabel("Iteration")
plt.ylabel("Cost")
```

```
plt.title("Min Prediction Cost function changing")

plt.show()
```



The final theta are:

1. theta 0 = 15209.70501
2. theta 1 = 7532.72664

The loss curves shows convergence as well, but here we can see there does some improvement the loss curve from traditional Model (the model without scaling years) are start with exponent 16, but here the scaling model loss curves start with the exponent 15 and the value of cost function at the last iteration is 65323174689302.125 which is smaller than the last one (544708182093170.06)

✓ Prediction value of minimum price from 2012-2018

Year-to-Price plot also shown at the end of this part

Here I also need to Scaling the years value from 2012 to 2018 to predict, and I cannot just direct use the scaling function I provide above because if I do so, the features will be centered around from 2012 to 2018, but the real center should be from 1998 to 2026

```

# a list to store the prediction value of minimum price
min_price_predict1 = []
# a list of scaling value for the prediction features
train_mean = np.mean(years)
train_std = np.std(years)
scaling_years1 = []
for year in predict_years:
    scaling_years1.append((year - train_mean)/train_std)

# process of prediction
for year in scaling_years1:
    min_price_predict1.append(linear_regression(year, theta0=min_final_theta01, t1

# build the data frame of the prediction result
predict_min_df = pd.DataFrame({
    "Years":predict_years,
    "Predicts":min_price_predict1
})

display(predict_min_df)

```

	Years	Predicts	
0	2012	15979.593480	
1	2013	16786.143309	
2	2014	17592.693138	
3	2015	18399.242968	
4	2016	19205.792797	
5	2017	20012.342627	
6	2018	20818.892456	

Next steps:

[Generate code with predict_min_df](#)[New interactive sheet](#)

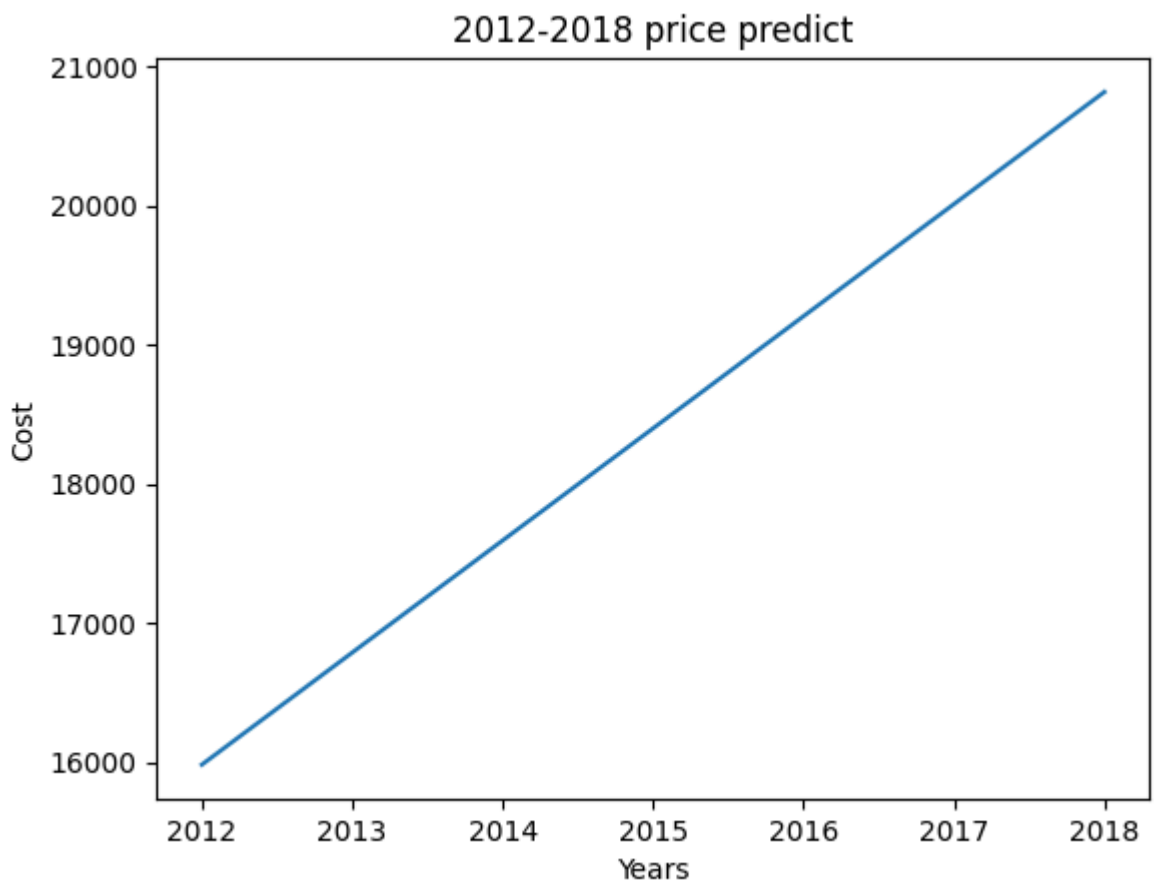
```

# draw the picture of prediction to see how the price changes
plt.plot(range(2012, 2019), min_price_predict1)

plt.xlabel("Years")
plt.ylabel("Cost")
plt.title("2012-2018 price predict")

plt.show()

```



✓ Maximum modeling

```
# max
max_price_col = 'max Price'
max_alpha_list = []
max_hypothesis_list1 = []
max_theta0_list1 = [theta[0]]
max_theta1_list1 = [theta[1]]
max_cost_list1 = []

# learning
dynamic_learning(learning_rate, data_df, iteration_num, theta, constant, max_p
                  max_alpha_list, max_hypothesis_list1, max_theta0_list1, 1

# a list to store the values of the learning rate during the iteration
max_final_hypothesis1 = max_hypothesis_list1[len(max_hypothesis_list1)-1]
# the value of theta 0 from the last iteration
max_final_theta01 = max_theta0_list1[len(max_theta0_list1)-1]
# the value of theta 1 from the last iteration
max_final_theta11 = max_theta1_list1[len(max_theta1_list1)-1]
# a list to store the error from the prediction max_final_hypothesis1
max_final_error1 = []

# calculate the errors of max prediction
for i in range(len(years)):
    max_final_error1.append(round((max_final_hypothesis1[i] - maxPrice[i]),5))
```

```
max_result_df1 = pd.DataFrame({
    "Years":years,
    "Max Price":maxPrice,
    "Predicts":max_final_hypothesis1,
    "Error":max_final_error1,
})
print(f"final modeling result (linear regression result):"+
      f" h(x) = {round(max_final_theta01,5)} + {round(max_final_theta11,5)} * x")
print(f"value of cost function of last prediction: {round(max_cost_list1[len(max_result_df1)-1],5)}")
display((max_result_df1))
```

final modeling result (linear regression result): h(x) = 30829.5139 + 12457.37851 * x
value of cost function of last prediction: 84481308473387.39

	Years	Max Price	Predicts	Error	
0	2026	57270	50776.572983	-6493.42702	
1	2025	56070	49442.727087	-6627.27291	
2	2024	55720	48108.881191	-7611.11881	
3	2023	40945	46775.035294	5830.03529	
4	2022	39730	45441.189398	5711.18940	
5	2021	39035	44107.343502	5072.34350	
6	2020	38675	42773.497606	4098.49761	
7	2019	38565	41439.651710	2874.65171	
8	2011	26070	30768.884541	4698.88454	
9	2010	25800	29435.038645	3635.03865	
10	2009	25805	28101.192749	2296.19275	
11	2008	24350	26767.346853	2417.34685	
12	2007	24425	25433.500957	1008.50096	
13	2006	26670	24099.655061	-2570.34494	
14	2005	26795	22765.809164	-4029.19084	
15	2004	26015	21431.963268	-4583.03673	
16	2003	25450	20098.117372	-5351.88263	
17	2002	25010	18764.271476	-6245.72852	
18	2001	24335	17430.425580	-6904.57442	
19	2000	19785	16096.579684	-3688.42032	
20	1999	19435	14762.733788	-4672.26621	
21	1998	19695	13428.887892	-6266.11211	

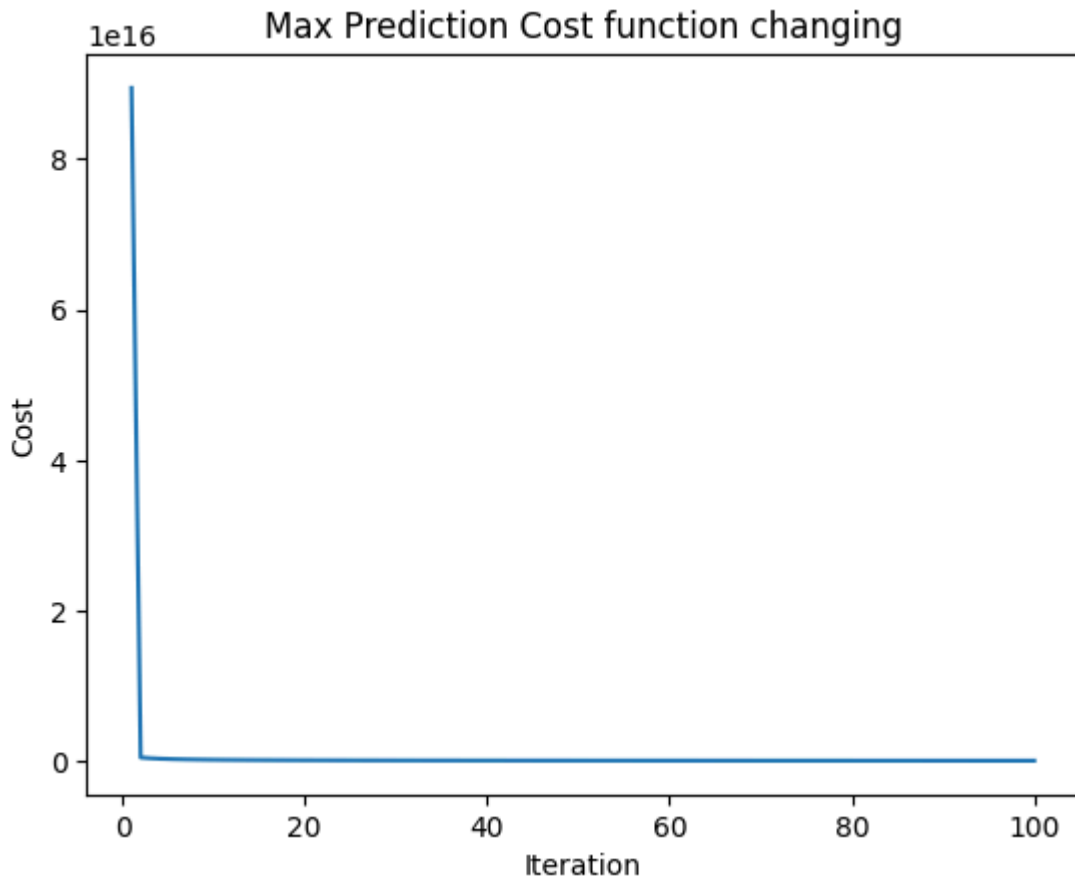
Next steps:

[Generate code with max_result_df1](#)[New interactive sheet](#)

```
#
# draw the figure of cost function to see how does the
# cost value changed during the 100 iterations
#
plt.plot(range(1, iteration_num + 1), max_cost_list1)

plt.xlabel("Iteration")
plt.ylabel("Cost")
plt.title("Max Prediction Cost function changing")

plt.show()
```



Here is the Loss curves of Maximum price is convergence faster than the Traditional Model, and even though the exponent is still start as 16 but the convergence is faster and the value of cost function (84481308473387.39) at the last iteration is also much smaller than the Traditional model (3777515622392022.5)

the final value of theta are:

1. Theta 0 = 30829.5139
2. Theta 1 = 12457.37851

✓ Prediction value of Maximum price from 2012 to 2018

```
max_price_predict1 = []
# scaling_years = feature_scaling(predict_years, 1)

for year in scaling_years1:
    max_price_predict1.append(linear_regression(year, theta0=max_final_theta01, t1

predict_max_df1 = pd.DataFrame({
    "Years":predict_years,
    "Predicts":max_price_predict1
})
display(predict_max_df1)
```

	Years	Predicts	
0	2012	32102.730437	
1	2013	33436.576333	
2	2014	34770.422229	
3	2015	36104.268126	
4	2016	37438.114022	
5	2017	38771.959918	
6	2018	40105.805814	

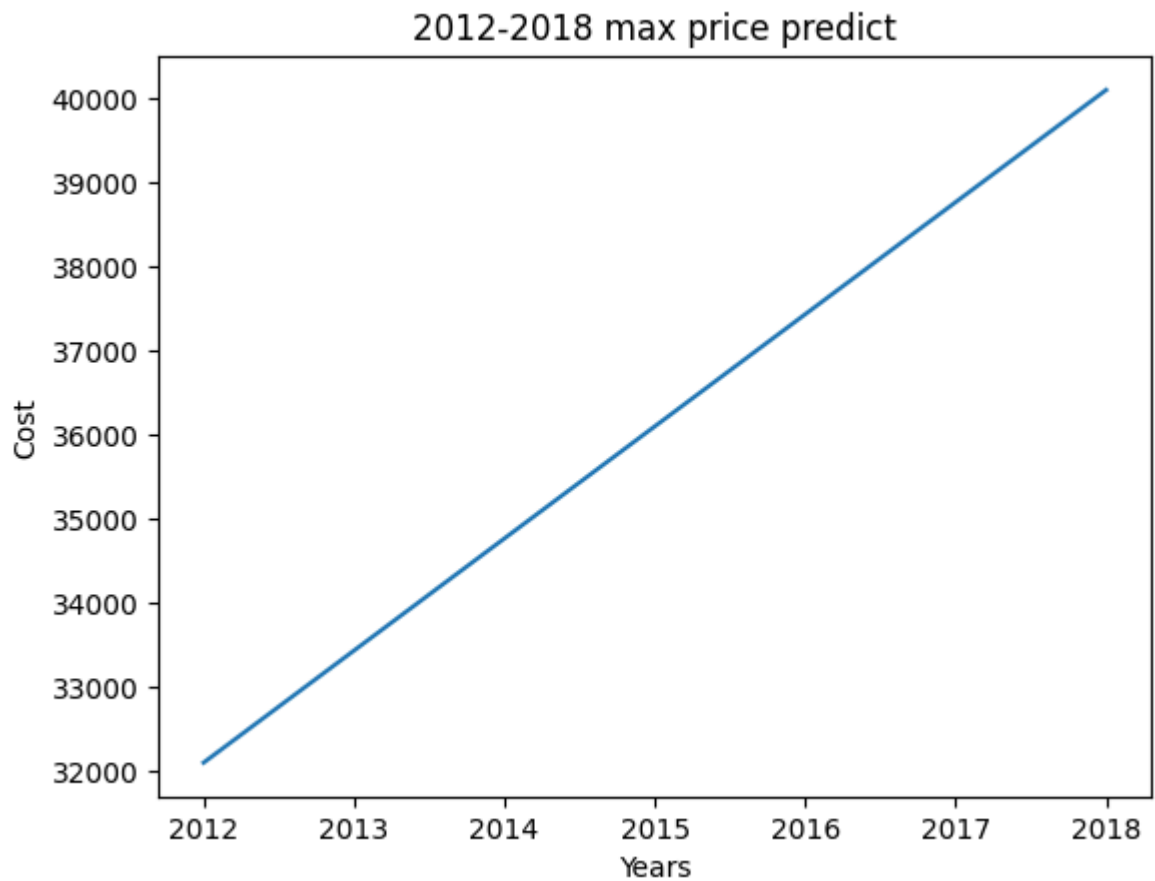
Next steps:

[Generate code with predict_max_df1](#)[New interactive sheet](#)

```
plt.plot(range(2012, 2019), max_price_predict1)

plt.xlabel("Years")
plt.ylabel("Cost")
plt.title("2012-2018 max price predict")
```

```
Text(0.5, 1.0, '2012-2018 max price predict')
```



✓ Year-to-Price Curves with predicted values

```
predict_dfl = pd.DataFrame({
    "year": predict_years,
    "min Price": min_price_predict1,
    "max Price": max_price_predict1
})
# add the prediction into the original frame
new_data_dfl = pd.concat(
    [data_df[["year", "min Price", "max Price"]], predict_dfl],
    ignore_index=True
)

new_data_dfl = new_data_dfl.sort_values(
    by="year"
).reset_index(drop=True)

display(new_data_dfl)
```

	year	min Price	max Price	
0	1998	11485.000000	19695.000000	
1	1999	11845.000000	19435.000000	
2	2000	11580.000000	19785.000000	
3	2001	11960.000000	24335.000000	
4	2002	12565.000000	25010.000000	
5	2003	13645.000000	25450.000000	
6	2004	14575.000000	26015.000000	
7	2005	14610.000000	26795.000000	
8	2006	14450.000000	26670.000000	
9	2007	13970.000000	24425.000000	
10	2008	14490.000000	24350.000000	
11	2009	16395.000000	25805.000000	
12	2010	17820.000000	25800.000000	
13	2011	18160.000000	26070.000000	
14	2012	15979.593480	32102.730437	
15	2013	16786.143309	33436.576333	
16	2014	17592.693138	34770.422229	
17	2015	18399.242968	36104.268126	
18	2016	19205.792797	37438.114022	
19	2017	20012.342627	38771.959918	
20	2018	20818.892456	40105.805814	
21	2019	24000.000000	38565.000000	
22	2020	24110.000000	38675.000000	
23	2021	24820.000000	39035.000000	
24	2022	25980.000000	39730.000000	
25	2023	27400.000000	40945.000000	
26	2024	32820.000000	55720.000000	
27	2025	33550.000000	56070.000000	
28	2026	33550.000000	57270.000000	

Next steps:

[Generate code with new_data_df1](#)[New interactive sheet](#)

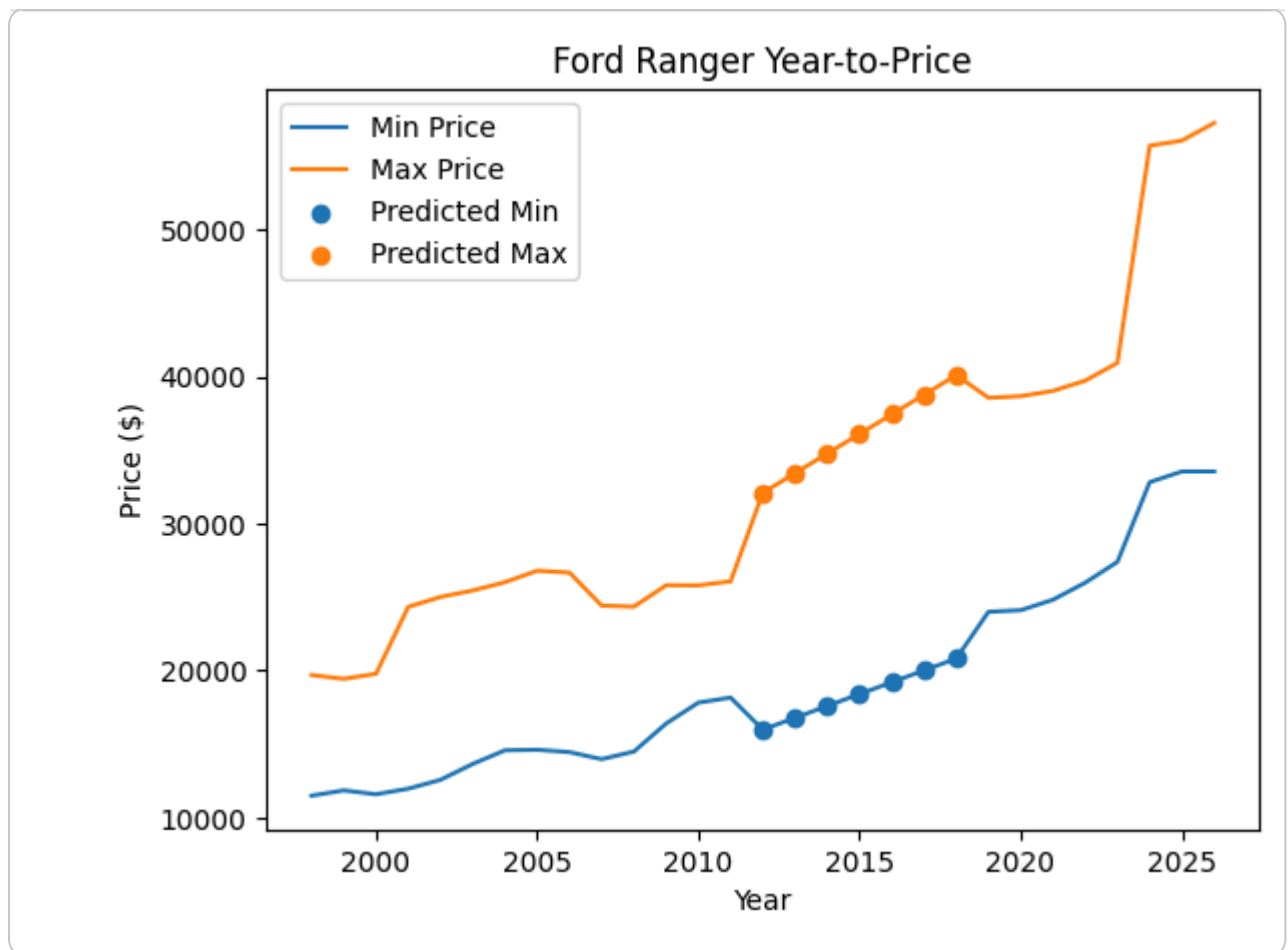
```
plt.plot(
    new_data_dfl["year"],
    new_data_dfl["min Price"],
    label="Min Price"
)

plt.plot(
    new_data_dfl["year"],
    new_data_dfl["max Price"],
    label="Max Price"
)

plt.scatter(
    predict_years,
    min_price_predict1,
    label="Predicted Min"
)

plt.scatter(
    predict_years,
    max_price_predict1,
    label="Predicted Max"
)

plt.xlabel("Year")
plt.ylabel("Price ($)")
plt.title("Ford Ranger Year-to-Price")
plt.legend()
plt.show()
```



Here is the prediction has a big change, the Prediction value from Traditional Model seems like flat, but here the prediction has an apparent slope

✓ Summary

The predicted price range of the 2026 Ford Ranger based on the traditional models is approximately 20,352.61 to 36,223.06. After applying feature scaling and a dynamic learning rate, the predicted price range increases to approximately 27,271.29 to 50,776.57.

Feature scaling and the dynamic learning rate improved the performance of the models. The loss curves converged more effectively, and the final costs were lower than those of the traditional models. However, the prediction errors are still relatively large. For the 2026 Ranger, the scaled minimum-price model has an error of approximately -

6,278.71, while the scaled maximum-price model has an error of approximately 6,493.43. Therefore, although the scaled models perform better than the traditional models, I do not think they are accurate enough for reliable real-world price prediction.

Based on these results, I would not make a decision to buy the next-generation Ranger based only on the prices predicted by these models, since an error of around

\$6,000 is significant when evaluating the price of a vehicle. One way to improve the models would be to include additional features instead of using only the model year. For example, vehicle specifications and other factors related to price could potentially help the models make more accurate predictions.

```
min_predict_2026 = min_final_hypothesis[0]
max_predict_2026 = max_final_hypothesis[0]

min_predict_2026_scaling = min_final_hypothesis1[0]
max_predict_2026_scaling = max_final_hypothesis1[0]

print(f"Tradition predict price range (Year = 2026): {round(min_predict_2026, 2)} - {round(max_predict_2026, 2)}")
print(f"Scaling predict price range (Year = 2026): {round(min_predict_2026_scaling, 2)} - {round(max_predict_2026_scaling, 2)}")
```

```
Tradition predict price range (Year = 2026): 20352.61 - 36223.06
Scaling predict price range (Year = 2026): 27271.29 - 50776.57
```

```
print("Unscaled Min:", min_final_theta0, min_final_theta1)
print("Unscaled Max:", max_final_theta0, max_final_theta1)

print("Scaled Min:", min_final_theta01, min_final_theta11)
print("Scaled Max:", max_final_theta01, max_final_theta11)

predict_compare = pd.DataFrame({
    "Years":predict_years,
```